Wrong the Same Way: Cross-Prompt Sharing Both Suppresses and Amplifies Reward Hacking in RLVR

Michael Li[*]
Carnegie Mellon University
ml7@andrew.cmu.edu

Rohan Pandey[*]
University of Washington
rpande@uw.edu

Ray Chen[*]
Independent
raychen256@gmail.com

Abstract

Two accounts of how reinforcement learning with verifiable rewards (RLVR) interacts with an imperfect verifier coexist in the literature: in one, the rate of reward hacking is inherited from the base model, while in the other, RL actively amplifies it. We study the two in a controlled RLVR environment small enough to compute the entire response distribution exactly. We vary the universality of a leak, or the fraction of prompts that accept the same wrong response, as well as the verifier's static false-positive rate. At its KL-regularized optimum, RLVR acts as a filter that reweights the verifier-accepted set as a whole without altering its composition, so the hack rate is inherited exactly. Taking that as a null model, a known decomposition writes the drift away from it as a sum over prompts of covariances between reward and the empirical tangent kernel. The terms from other prompts vanish without parameter sharing. With sharing, every prompt that accepts a wrong answer pushes it up and every prompt that rejects it pushes it down through the baseline, in proportion to the model's probability of producing it there. A shared-parameter policy therefore drives a leak that few prompts accept below the base rate and amplifies one that most prompts accept. At every convergent learning rate under Adam, SGD and sampled GRPO, hacking ends higher at high universality than at low, but whether either end crosses the base rate depends on the step size. Replacing the shared architecture with independent per-prompt copies removes the dependence. The direction of this push depends on neither the verifier's error rate nor universality alone, but on how much of the model's probability of a wrong answer, weighted by that kernel, falls on prompts that accept it, which experiments with a frozen base model support. Verifier fixes should therefore target the mistakes the model repeats across prompts: two patches that remove equally many errors differ by up to 38 points of true accuracy. We propose this alignment as a hypothesis for why earlier studies disagree.

1 Introduction

In reinforcement learning with verifiable rewards (RLVR), a response is rewarded when a programmatic verifier accepts it [Shao et al., 2024, Guo et al., 2025]. Because verifiers are imperfect proxies for correctness, policies are vulnerable to *reward hacking*: optimizing for the verifier's acceptance criteria rather than the intended task objective [Skalse et al., 2022]. Generative judges accept non-word symbols and generic reasoning openers [Zhao et al., 2026], test suites accept programs that pass only the visible tests [Zhang, 2026], and outcome verifiers reward a correct final answer reached by invalid reasoning [Yuan et al., 2026], one reason process supervision [Lightman et al., 2023] was proposed over outcome supervision [Cobbe et al., 2021]. Every such false positive provides an exploitable shortcut where the policy is rewarded without being correct.

[*]Equal contribution. Our source code is available at `github.com/Rohan-Pandey1729/RLVR-Reward-Hacking`.

40th Conference on Neural Information Processing Systems (NeurIPS 2026). Workshop: Transitioning from Pre-Training to Post-Training.

Two accounts of how RL interacts with these false positives coexist. In the first, RLVR is robust to them and any hacking is inherited from the base model: verifier noise up to 20% leaves validation reward within one point of clean training [Plesner et al., 2026], noise rescales convergence time rather than the destination while the verifier's Youden index stays positive [Rad et al., 2026], and a preregistered comparison of leaky and hardened test suites suggests that the wrong programs a trained model emits were already emitted by the base model [Zhang, 2026]. In the second, RLVR is vulnerable to them and actively amplifies hacking: RL amplifies hacking seeded by a trace of contamination [Khalifa et al., 2026], systematic verifier error produces delay, plateau or collapse [Egashira et al., 2026], models learn to game extensional verifiers [Helff et al., 2026], and production reward hacking generalizes into broader misalignment [MacDiarmid et al., 2025]. The studies behind these accounts use diverse setups, so they do not identify what decides the outcome.

Theory sides with the first account. At the KL-regularized optimum of the RLHF objective [Stiennon et al., 2020, Ouyang et al., 2022] the policy is $\pi^\star(y \mid x) \propto \pi_{\mathrm{ref}}(y \mid x)\, e^{r(x,y)/\beta}$, so within any reward level set the odds between responses are those of the reference model [Korbak et al., 2022]. RLVR thus acts as a filter on the accepted set: it changes how much mass the set receives, not its composition, which is consistent with RLVR sharpening rather than expanding a base model's abilities [Yue et al., 2025, Gai et al., 2025, Kruszewski et al., 2026, Dymetman, 2026]. For verifier false positives, this predicts inheritance. However, training in practice takes first-order steps on a model whose parameters are shared across prompts, often with little or no KL term; we ask what happens to verifier false positives under such training.

We answer the question in a controlled environment (Figure 1): single-digit multiplication with a two-digit answer space, in which the full response distribution of a small transformer is enumerated exactly at every step. The verifier accepts the correct answer plus designated wrong strings, and we vary how many prompts share the same wrong string, its *universality*, at a fixed false-positive rate. Our contributions are as follows:

- **We identify a two-sided effect of parameter sharing.** Sharing suppresses hacks shared among few prompts and amplifies hacks shared among many prompts. Hacking rises with universality at every convergent learning rate under three training setups, though the level depends on the step size, and per-prompt copies of the model remove the effect (Sections 6.1 and 6.2).
- **We identify what sets the direction.** By a known decomposition, other prompts affect a hack only through shared parameters. With aligned gradients, the net update to that string pushes it up when the kernel-weighted share of its probability on accepting prompts exceeds the model's pass rate. Frozen base model controls support this (Sections 4 and 6.3).
- **We identify what to audit.** Two verifier patches that remove equally many errors differ by up to 38 points of true accuracy over 10 seeds (Section 6.5). Audits should check whether accepted wrong answers are ones the model already produces across prompts.

2 Background

Systematic error and amplification. Egashira et al. [2026] established the premise we build on: whether training plateaus or collapses depends on the "specific pattern of introduced errors" rather than on the blanket error rate. Khalifa et al. [2026] showed that about 1% of hacking demonstrations in supervised fine-tuning primes hacking under RLVR, and Zhao et al. [2025] showed that RL post-training amplifies behaviors already present in pretraining; Zhang [2026] reports inheritance while Khalifa et al. [2026] and Nishimura-Gasparian et al. [2026] report amplification and specification gaming. Work on what corrupted rewards can do [Everitt et al., 2017, Skalse et al., 2022, Gao et al., 2023] and on group-relative advantage estimators [Yang et al., 2026, Zeng et al., 2026] is orthogonal.

Learning dynamics and coherent gradients. The decomposition in Section 4 follows Ren and Sutherland [2025] for fine-tuning and Tomihari [2026] for RL post-training. The mutual reinforcement of gradients from examples that agree is the coherent-gradients hypothesis of Chatterjee [2020]. We do not claim the decomposition; what we use in our work is the push-down: a rewarded response on one prompt is penalized through the baseline on every prompt that rejects it, so the mechanism that amplifies a widely accepted wrong answer suppresses a narrowly accepted one. Appendix F covers further related work.

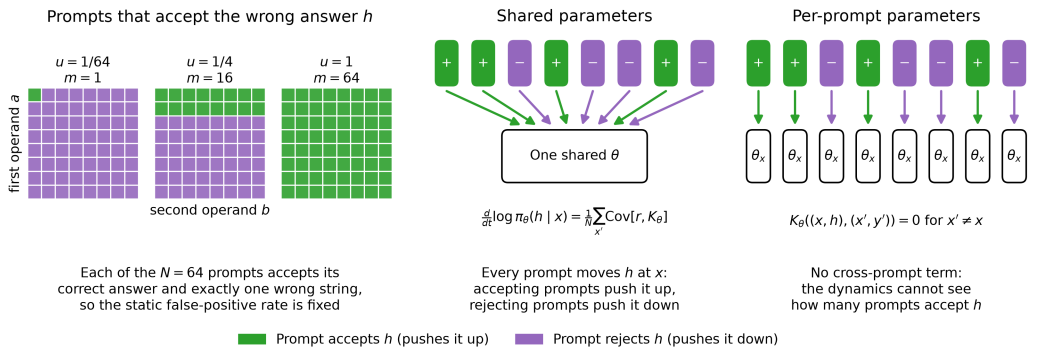


Figure 1: The environment and the mechanism. Left: the $N = 64$ prompts, colored by whether the verifier accepts a designated wrong answer h; every prompt accepts exactly one wrong string, so sweeping $u = m/N$ leaves the static false-positive rate unchanged. Center and right: with shared parameters every prompt's gradient lands on the same weights, so h is pushed up by the prompts that accept it and down by those that reject it (Lemma 1); with per-prompt parameters the cross-prompt term is identically zero (Corollary 1).

3 Setup: RLVR with an Exploitable Verifier

Let $\mathcal{X}$ be a finite prompt set with $|\mathcal{X}| = N$ and where each prompt is equally weighted, and let $\mathcal{Y}$ be a finite response space. A response is correct if $c(x, y) = 1$; we refer to the correct response as g_x, which is unique in our setting. The deployed verifier returns $r(x, y) \in \{0, 1\}$ with $r \geq c$, so it has no false negatives. $\mathcal{A}(x) = \{y : r(x, y) = 1\}$ is the accepted set and $\mathcal{H}(x) = \{y : r(x, y) = 1,\ c(x, y) = 0\}$ its false-positive part, or the *leaks*. RLVR maximizes $J(\theta) = \mathbb{E}_x\, \mathbb{E}_{y\sim\pi_\theta(\cdot|x)}[r(x, y)]$.

Definition 1 (Hack fraction)**.** The hack fraction is $\rho(\pi) = \frac{1}{N}\sum_x \pi(\mathcal{H}(x) \mid x) \,/\, \pi(\mathcal{A}(x) \mid x)$, the probability that a rewarded response is a false positive, averaged over prompts. We also track the prompt-averaged log-odds $\Lambda(\pi) = \frac{1}{N}\sum_x \log\left[\pi(\mathcal{H}(x) \mid x) \,/\, \pi(g_x \mid x)\right]$. Both are finite under any softmax parameterization if no $\mathcal{H}(x)$ is empty.

Definition 2 (Leak universality)**.** For a response h, let leak count $m(h) = |\{x : h \in \mathcal{H}(x)\}|$ and leak universality $u(h) = m(h)/N$. A leak with $u = 1$ is accepted on every prompt. A leak with $u = 1/N$ is accepted on one prompt only, which we call *idiosyncratic*.

3.1 The filter baseline

Proposition 1 (RLVR is a filter; known)**.** *Assume $r \geq c$ and let $\pi_{\mathrm{ref}}(\cdot \mid x)$ have full support. Let $\pi^\star_\beta = \arg\max_\pi J(\pi) - \beta\, \mathbb{E}_x \mathrm{KL}(\pi(\cdot \mid x) \| \pi_{\mathrm{ref}}(\cdot \mid x))$. Then $\pi^\star_\beta(y \mid x) \propto \pi_{\mathrm{ref}}(y \mid x)\, e^{r(x,y)/\beta}$, so for any y, y' with $r(x, y) = r(x, y')$ the odds are unchanged. Consequently $\rho(\pi^\star_\beta) = \rho(\pi_{\mathrm{ref}})$ and $\Lambda(\pi^\star_\beta) = \Lambda(\pi_{\mathrm{ref}})$ for every $\beta > 0$. The same invariance holds at every step of any multiplicative update $\pi_{t+1}(\cdot \mid x) \propto \pi_t(\cdot \mid x) \exp(\eta f(r(x, \cdot)))$.*

The closed form is the Gibbs variational principle, standard in RL [Peters et al., 2010] and in the LLM setting [Korbak et al., 2022, Kruszewski et al., 2026, Dymetman, 2026], and it is the baseline every departure below is measured against. As $\beta \to 0$, $\pi^\star_\beta \to \pi_{\mathrm{ref}}(\cdot \mid \mathcal{A}(x))$: along the whole regularization path the accepted set keeps the reference model's composition, so nothing inside it can move. The pooled share $\sum_x \pi(\mathcal{H}(x) \mid x) / \sum_x \pi(\mathcal{A}(x) \mid x)$ that prior work reports is a weighted average of these invariant per-prompt ratios with weights that RLVR moves, so it drifts under pure filtering (by +0.057 in our environment; Appendix A); we report per-prompt statistics throughout.

4 Why First-Order Training Departs From Filter

In practice, RLVR takes first-order steps on a shared-parameter model, often with a small or absent KL term. The following decomposition, following Ren and Sutherland [2025] for fine-tuning and Tomihari [2026] for RL post-training, describes the resulting drift exactly.

Lemma 1 (Drift decomposition; known)**.** *Under exact-gradient flow $\dot{\theta} = \nabla_\theta J(\theta)$,*

$$\frac{d}{dt}\log \pi_\theta(y \mid x) = \frac{1}{N}\sum_{x' \in \mathcal{X}} \mathrm{Cov}_{y' \sim \pi_\theta(\cdot \mid x')}\Big[\, r(x', y'),\ K_\theta\big((x, y), (x', y')\big)\Big], \tag{1}$$

where $K_\theta((x, y), (x', y')) = \langle \nabla_\theta \log \pi_\theta(y \mid x), \nabla_\theta \log \pi_\theta(y' \mid x')\rangle$ is the empirical tangent kernel of the log-policy (Appendix A).

The $x' = x$ term is the ordinary within-prompt update, present for any policy; the $x' \neq x$ terms are the contribution of parameter sharing. Most of our experiments use Adam [Kingma and Ba, 2015], whose momentum makes the update history-dependent, so the identity does not apply verbatim; the support argument below does, because Adam's state is per-coordinate, and we also report plain SGD. The kernel is the finite-width empirical kernel, not the infinite-width limit of Jacot et al. [2018].

Corollary 1 (Per-prompt parameters cannot see universality)**.** *If $\theta = (\theta_x)_{x \in \mathcal{X}}$ and $\pi_\theta(\cdot \mid x)$ depends on θ_x only, then $K_\theta((x, y), (x', y')) = 0$ for $x' \neq x$,* (1) *collapses to its $x' = x$ term, and the trajectory of each block θ_x is an autonomous ODE that depends on the verifier only through $r(x, \cdot)$. Such a policy can depend on m only through what it inherited at initialization.*

A per-prompt policy is therefore a control rather than a baseline. It does not sit *on* the filter baseline: for two equally rewarded responses y_1, y_2 at a prompt x, gradient flow gives $\frac{d}{dt}\log(\pi_1/\pi_2) = \frac{1}{N}(\pi_1 - \pi_2)(1 - V(x))$ with $\pi_i = \pi_\theta(y_i \mid x)$ and pass rate $V(x) = \sum_y \pi_\theta(y \mid x)\, r(x, y)$, the rich-get-richer behavior of softmax policy gradient [Mei et al., 2020, Lochab et al., 2026]. That drift is prompt-local and blind to u, which is what a control requires.

The quantity that matters. Equation (1) does not isolate $u = m/N$. Specializing it to a leak h accepted on the set U, the acceptance fraction that appears is kernel- and policy-weighted:

$$u_{\mathrm{eff}}(x, h) \;=\; \Big(\textstyle\sum_{x' \in U} w_{x'}\Big) \Big/ \Big(\textstyle\sum_{x'} w_{x'}\Big), \qquad w_{x'} = K_{\theta, xx'}\, \pi_\theta(h \mid x'), \tag{2}$$

where $K_{\theta,xx'} = K_\theta((x, h), (x', h))$ couples h at x to the same string at x'. The weights can be negative in general (13% of the entries are at our base models; Appendix C); when they are positive, u_{eff} is the share of the model's kernel-weighted mass on h that sits on prompts which accept h. The raw fraction m/N is recovered only when the kernel entries and the probabilities are the same at every prompt.

Definition 3 (Coherent kernel model)**.** Write $K_\theta = K_\theta^{\mathrm{coh}} + R$ with $K_\theta^{\mathrm{coh}}((x, h), (x', y')) = \kappa_{xx'}\, \mathbf{1}[y' = h] + a(x, x') + b(x, x', y')$, where a does not depend on y' and b does not depend on h. The a and b terms drop out of (1) and of the within-prompt log-odds respectively (Appendix A), leaving the same-string entries $\kappa_{xx'}$ and a residual R that we do not bound. Coherence is a property of how a response is computed, not of string identity: a wrong answer produced by prompt-independent machinery is coherent, while correct answers that coincide on two prompts need not be.

Proposition 2 (Universality threshold)**.** *Let h be accepted on U, let the correct answer be incoherent across prompts ($\kappa^g_{xx'} = 0$ for $x' \neq x$) with self-scale $\kappa_{s,g}$, and let the pass rate be the same on every prompt, $V(\cdot) \equiv \bar{V}$, which makes $\bar{V} = J(\theta)$ the policy's verifier pass rate. Then at any prompt x the coherent part of the kernel contributes*

$$\frac{d}{dt}\Big[\log \pi_\theta(h \mid x) - \log \pi_\theta(g_x \mid x)\Big] = \frac{W_x}{N}\left(u_{\mathrm{eff}}(x, h) - \bar{V}\right) \;-\; \frac{1 - \bar{V}}{N}\,\kappa_{s,g}\,\pi_\theta(g_x \mid x), \tag{3}$$

with $W_x = \sum_{x'} \kappa_{xx'}\, \pi_\theta(h \mid x')$ and u_{eff} taken with $\kappa_{xx'}$ in place of $K_{\theta,xx'}$. If in addition $x \in U$, $\pi_\theta(h \mid \cdot) \equiv p_h$ and $\kappa_{xx'}$ equals κ_s on the diagonal and $\kappa > 0$ off it, then with $p_g = \pi_\theta(g_x \mid x)$ this is $\kappa\, p_h\,(u - \bar{V}) + \frac{1-\bar{V}}{N}\big[(\kappa_s - \kappa)\, p_h - \kappa_{s,g}\, p_g\big]$, whose leading-order sign is that of $u - \bar{V}$ (Appendix A).

Three consequences follow. The verifier's false-positive rate does not appear: accepting more distinct wrong responses per prompt changes every static audit metric and leaves (3) unchanged. Sharing is two-sided: every prompt that rejects a wrong answer contributes a negative advantage, so the term that amplifies a widely accepted wrong answer pushes a narrowly accepted one down, and a per-prompt policy has no such term. And the threshold rises during training with $\bar{V}$, so a wrong answer can be amplified early and suppressed later. Proposition 2 describes one channel of the drift, the same-string channel, at leading order under homogeneity assumptions that our base models violate; it does not predict where the final drift crosses zero, and raw u is not the controlling variable in our headline sweep, where the supervised stage concentrates a leak's mass on the prompts that accept it (Section 6.3; Appendix C compares the channel with the exact cross-prompt drift).

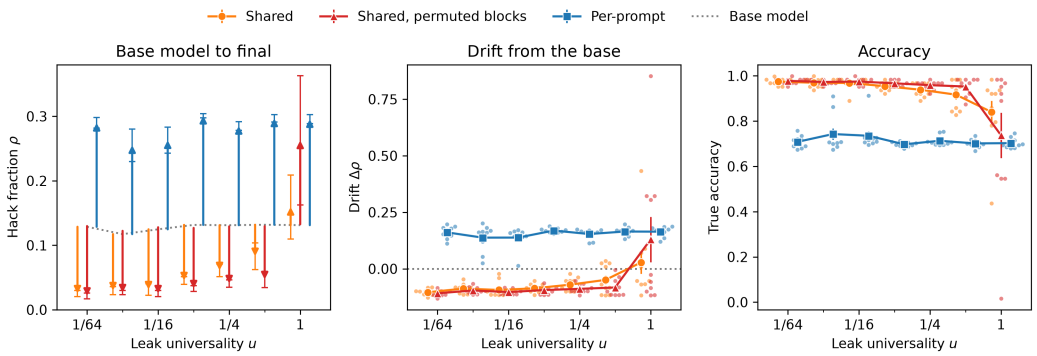


Figure 2: At a fixed false-positive rate, the more prompts share the wrong answer, the less the shared policy suppresses it, and at $u = 1$ the suppression disappears. Left: arrows from each condition's base model to its mean final hack fraction. Center: drift from the base model, zero under the filter baseline. Right: true accuracy. Dots are ten seeds, markers their means ± 1 s.e.m. at the operating point selected by verifier reward, and *permuted blocks* randomizes block membership.

5 Experiments

Task and verifier. We use single-digit multiplication as the task. $\mathcal{X}$ consists of $N = 64$ pairs (a, b) where $a, b \in \{2, \ldots, 9\}$. The correct response is the two-digit decimal string of $a \times b$, and the response space is all 100 two-digit strings ranging from "00" to "99", so that $\pi_\theta(\cdot \mid x)$ is enumerable for every prompt at every step. Note that there exist response strings that can never be the correct answer to any input (a, b). The verifier accepts the correct string plus k designated wrong strings per prompt, none of which is the correct answer of any prompt; prompts are partitioned into blocks of size m whose members share the same leak strings, so varying m sets $u = m/N$ at a fixed static false-positive rate $k/(|\mathcal{Y}| - 1)$, and varying k sweeps the false-positive rate at fixed universality.

Base model. The policy is a 2-layer, 64-dimensional transformer [Vaswani et al., 2017] with 101,706 parameters, trained from scratch on a supervised target. At prompt x, the target gives the correct answer with probability λ_x and a uniformly random string otherwise, and a share $\varepsilon = 0.03$ of it is replaced by leak demonstrations, since a trace of contamination suffices to seed hacking [Khalifa et al., 2026]. The λ_x are evenly spaced over $[0, 0.9]$, so the model solves some prompts most of the time and others almost never, and a fixed permutation assigns them to prompts independently of the blocks. The base model has accuracy 0.441 and hack fraction 0.128. Its hack fraction varies by 0.015 across conditions, so we report hacking as a drift from each run's own base model, $\Delta\rho$ for the hack fraction and $\Delta\Lambda$ for the log-odds, which Proposition 1 makes exactly zero under the filter baseline (verified to floating-point precision in Appendix B).

Per-prompt control. The transformer is paired with a tabular policy of free per-prompt parameters under the same autoregressive factorization, initialized to reproduce the base transformer's distribution to 10^{-4}; both are trained on the same objective and learning-rate grid (Table 1).

Protocol. Unless stated otherwise we run exact-expectation policy gradient with the exact mean baseline, the expectation limit of RLOO [Ahmadian et al., 2024] and Dr. GRPO [Liu et al., 2025], for 400 Adam steps at ten seeds per cell, and report the operating point a practitioner would select without ground truth: the smallest learning rate whose final verifier reward is within 10^{-3} of the best.

6 Results

Our main results are demonstrated in Figure 2: at a fixed false-positive rate, universality sets the direction for the shared-parameter policy. With the static false-positive rate fixed and only the number of prompts sharing the accepted wrong response varied, the shared-parameter policy's hack fraction climbs from 0.025 ± 0.005 at $u = 1/64$ to 0.160 ± 0.050 at $u = 1$ (10 seeds), against a base value of 0.128: as a drift, $\Delta\rho$ moves from -0.104 ± 0.004 to $+0.028 \pm 0.050$, and true accuracy falls from 0.974 to 0.840. The per-prompt policy, initialized identically and trained on the same grid, is flat:

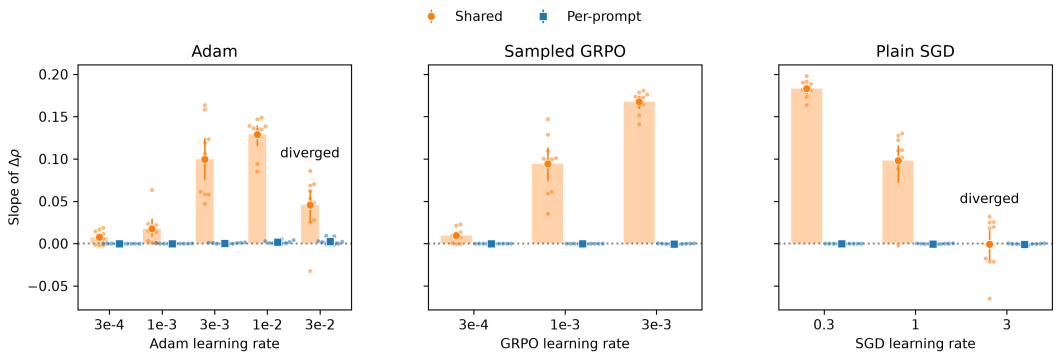


Figure 3: The universality slope of $\Delta\rho$ (per doubling of m) at every learning rate, for both policies under three training setups. Bars and markers are means over ten seeds, dots are seeds and whiskers bootstrap 95% intervals. The shared slope is positive everywhere except where SGD diverges; the per-prompt slope is near zero everywhere.

regressing $\Delta\rho$ on $\log_2 m$ gives $+0.0028$ per doubling against $+0.0177$ for the shared policy. It does not sit on the filter baseline, but its deviation carries no information about universality, as Corollary 1 requires. Randomizing block membership reproduces both the suppression and the slope ($+0.0270$ against -0.0001 for the per-prompt arm), so contiguous blocks coinciding with the first operand are not the cause.

The $u = 1$ end under Adam is heterogeneous: the mean drift at the selected operating point is within its own error bar, the ten final hack fractions span 0.02, 0.06, 0.06, 0.08, 0.08, 0.14, 0.17, 0.20, 0.22, 0.56, with one collapsed seed and a median (0.11) below the base value, and 16% of prompts finish with the leak more probable than the correct answer against 2% at $u = 1/64$, which is why $\Delta\rho$ approaches zero while $\Delta\Lambda$ stays negative (-9.0 to -3.7). A decisive sign change into amplification appears under sampled GRPO, under plain SGD (Section 6.4) and under Adam at the next learning rate up (Section 6.1). Crossing universality with the static false-positive rate, both swept eightfold at a fixed total base hack mass, attributes 43% of the variance in $\Delta\Lambda$ to universality and 12% to the false-positive rate (Figure 7, Appendix D): the false-positive rate is the weaker variable.

6.1 The direction holds at every convergent learning rate but the level does not

Figure 3 reports the universality slope at every learning rate, for three training setups and both arms. The shared-parameter slope is positive with a bootstrap interval excluding zero at 10 of 11 settings, the exception being the largest SGD step, where training diverges, and the per-prompt slope is within a few thousandths of zero at every setting: cross-prompt sharing adds a component to the drift that grows with universality at every convergent step size we tried.

The level is not robust. Under Adam, all ten seeds suppress the idiosyncratic leak at the three smallest learning rates and all ten amplify it at the two largest, where accuracy on that condition also collapses (0.275 at 10^{-2}); the per-prompt arm's level moves with step size in the same way, without a slope, so that movement is prompt-local. Three Adam learning rates saturate the verifier at $u = 1$ to within the rule's tolerance; the rule picks the smallest, 10^{-3}, where $\Delta\rho$ at $u = 1$ is $+0.028$, whereas at 3×10^{-3} it is $+0.600 \pm 0.084$ with 10 of 10 seeds positive while the idiosyncratic leak is still suppressed in every seed (-0.098, accuracy 0.930). The suppression claim therefore carries a step-size qualifier, and the missing amplification at $u = 1$ in Figure 2 reflects the conservative tie-break rather than the optimizer (Table 2).

6.2 Sharing is the channel

Lemma 1 splits the drift into an own-prompt term and a cross-prompt term that is the entire contribution of sharing (Figure 4, top). By autograd at the base model (three seeds) the cross-prompt term is exactly 0 for the per-prompt policy at every universality, where Corollary 1 is exact, and for the shared policy it rises from 6.6 ± 1.1 to 12.2 ± 0.8. The cross-prompt gradient cosine of each prompt's own leak rises from $+0.02$ to $+0.29$ across the sweep, against $+0.03$ for correct answers.

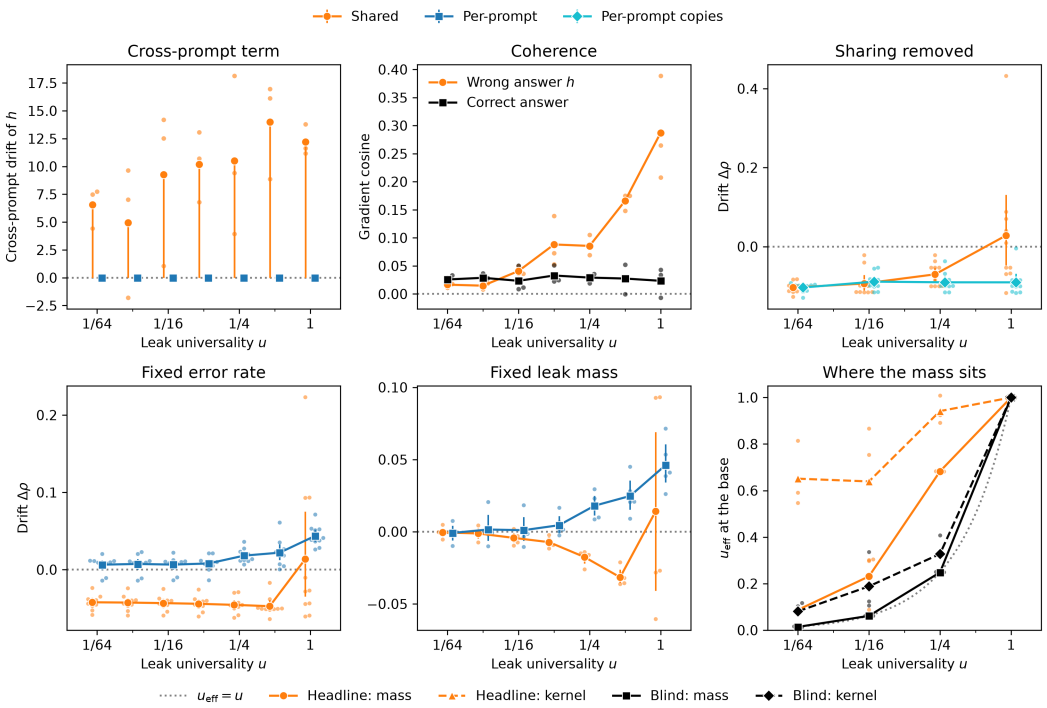


Figure 4: Sharing is the channel (top), and the base model decides where the mass sits (bottom). Top: the cross-prompt component of the exact drift of $\log \pi_\theta(h \mid x)$ at the base model (Lemma 1), zero for per-prompt parameters; the cross-prompt gradient cosine of each prompt's own leak and of its correct answer; and the shared transformer against per-prompt copies of the identical architecture. Bottom: one frozen base per seed with only the verifier's mask varying, holding either the static false-positive rate or the accepted string's per-prompt base mass fixed; and u_{eff} (Equation 2) at the base model against raw u, in its mass-only and kernel-weighted readings, for the headline and the frozen blind bases. Dots are seeds; whiskers in the drift panels are bootstrap 95% intervals.

The tabular comparison confounds parameterization with sharing. The cleanest test gives each prompt its own copy of the transformer, all copies starting from the same checkpoint and taking the same number of steps, with copy x evaluating only prompt x. The dependence of $\Delta\rho$ on universality disappears: the slope falls to $+0.0019$ per doubling (CI $[-0.0008, +0.0050]$) against $+0.0210$ for the shared policy under matched conditions, a paired within-seed difference of $+0.0191$ (CI $[+0.0082, +0.0330]$, 10 seeds), and at $u = 1$ the copies drift -0.090 and keep 0.959 accuracy where the shared policy loses both. This is Corollary 1 with the architecture held fixed. The hack fraction cannot show what sharing adds at low u, where both arms sit at the floor set by the few prompts the leak wins, but the log-odds can: at $u = 1/64$ the paired difference in $\Delta\Lambda$ between the shared policy and the copies is -1.00 (CI $[-1.56, -0.36]$), and at $u = 1$ it is $+2.11$ (CI $[+0.75, +3.85]$).

6.3 The base model determines where the mass sits

In the sweeps above, changing the verifier also changes the supervised target, so verifier universality and the universality of the base model's own error modes coincide by construction. To separate them we freeze one checkpoint per seed, the *blind base*, whose supervised target carries a shared string h and a per-prompt string s_x on every prompt, identically in every condition, and vary only which prompts the verifier accepts h on; one variant accepts s_x on the remaining prompts, which fixes the static false-positive rate, and the other accepts nothing wrong there, which fixes the accepted string's per-prompt base mass (Appendix B).

Read per universality value, the shared policy's curve is a step, not a line (Figure 4, bottom). In the fixed-rate variant the shared policy suppresses the accepted string far more than the per-prompt policy at every $u < 1$, with a paired within-seed difference of -0.049 at $u = 1/64$ (CI $[-0.057, -0.038]$) and -0.069 at $u = 1/2$ (CI $[-0.079, -0.060]$), excluding zero at all 6 conditions below $u = 1$. At

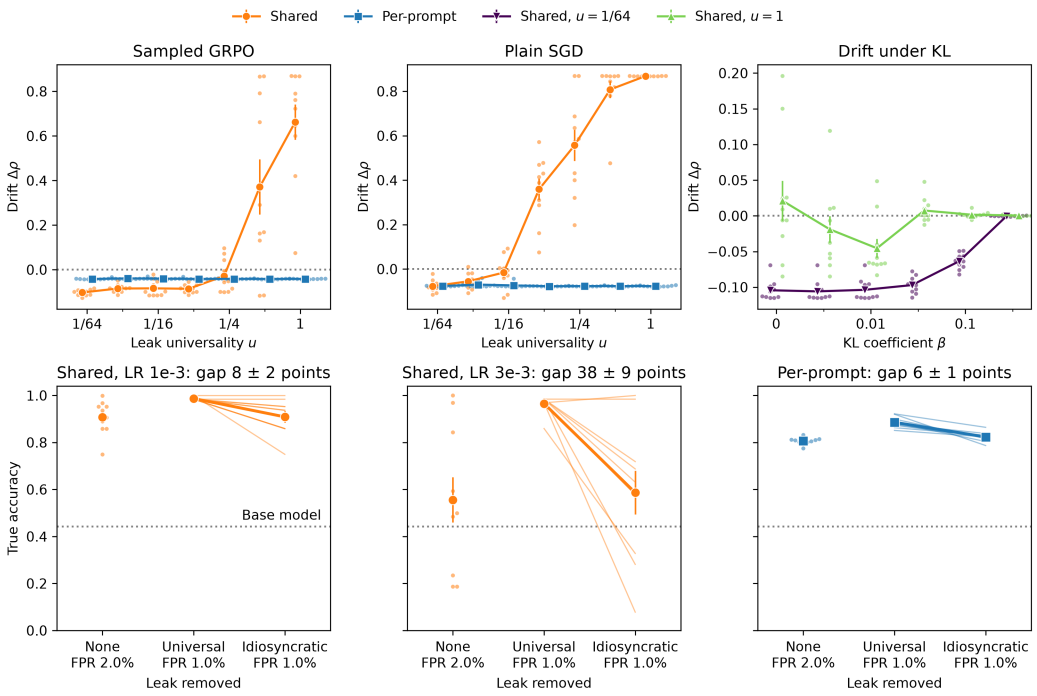


Figure 5: The effect in practice. Top: sampled GRPO (group size 8), plain SGD, and the drift at the two ends of the universality sweep as the KL coefficient β grows (Adam, operating point); dots are seeds and markers their means ± 1 s.e.m. Bottom: two verifier patches with identical static audit value buy very different true accuracy; thin lines join one seed's two patched runs from the same checkpoint, titles give the paired gap with its s.e.m., and the dotted line is the base accuracy.

$u = 1$, where no prompt rejects h, the shared policy's mean drift is within its own error bar and the per-prompt policy amplifies it, and at the next learning rate up the shared policy amplifies the blind base's string about as strongly as the headline sweep's ($+0.439$ against $+0.600$ at the same rate). The fixed-mass variant agrees wherever the accepted string carries enough base mass to move (-0.056 at $u = 1/2$, CI $[-0.064, -0.048]$; $+0.427$ at $u = 1$ at the higher rate).

The two variants therefore agree, and their difference from the headline sweep is what Equation (3) predicts. In the blind base the model's mass on h is spread nearly evenly over prompts, so the mass-only reading of u_{eff} is close to u (Figure 4, bottom right): every rejecting prompt pushes h down, and everything below $u = 1$ ends suppressed once $\bar{V}$ has risen. In the headline sweep the supervised stage puts the leak's mass on the prompts that accept it and only the uniform floor elsewhere, so the same reading is well above u (0.09, 0.23, 0.68 and 1.00 at $u = 1/64, 1/16, 1/4$ and 1; the kernel-weighted reading separates the two bases likewise), the push-down from rejecting prompts is weak, and the drift rises gradually with u. What the base model contributes is not a switch that enables sharing but the placement of its own mass; the two bases represent h at $u = 1$ with similar cross-prompt coherence ($+0.255$ against $+0.294$; Appendix C).

6.4 Robustness across GRPO, SGD, and KL penalties

The effect is not an artifact of exact-expectation gradients or of Adam (Figure 5, top). Under sampled GRPO [Shao et al., 2024] with group size 8 and standard-deviation normalization, $\Delta\rho$ moves from -0.103 ± 0.004 to $+0.663 \pm 0.079$ over 10 seeds, crossing zero at $u = 0.27$. Under plain SGD, the optimizer the analysis assumes, the shared policy's hack fraction climbs from 0.051 to 1.000 and its accuracy falls from 0.926 to 0.000, crossing at $u = 0.07$, while the per-prompt policy stays below its base model throughout.

A KL penalty pulls the policy back toward the filter baseline, as Proposition 1 requires, and here that costs accuracy: at $u = 1/64$, raising β to 0.3 takes $\Delta\rho$ from -0.104 to 0.000, removing the beneficial suppression, and accuracy from 0.978 to 0.821 (10 seeds; Figure 9). KL is not a free defense: it

commits the model to the base model's composition of the accepted set, which helps only when that composition is better than what training would reach; Ackermann et al. [2026] reach a compatible conclusion from the other direction.

6.5 Budgeting verifier hardening

We contaminate a base model with one universal and one idiosyncratic leak string per prompt and compare two patches that remove the same number of accepted wrong (prompt, response) pairs, so that a static audit rates them equally (Figure 5, bottom). Both patches start from the same checkpoint within each seed, so every difference arises during RL, and the verifier's own reward stays at 0.973 or above under both patches at both learning rates, so neither would be flagged during training. At the conservative operating point, 10^{-3}, the universal patch leaves the shared policy at 0.987 true accuracy and the idiosyncratic patch at 0.909, a paired within-seed gap of 8 ± 2 points (bootstrap CI $[4, 13]$, 10 seeds); at 3×10^{-3}, where the universal leak is amplified, the gap is 38 ± 9 points (CI $[22, 55]$, positive in 9 of 10 seeds). The per-prompt policy shows a gap of 6 points (CI $[5, 8]$), which can only come from prompt-local dynamics acting on the base model's unequal mass on the two families; if the two effects add, the remaining 32 points at the larger rate (CI $[16, 48]$) require sharing, whereas at 10^{-3} the remainder is 2 points (CI $[-3, 7]$). A patch that removes the error the model shares across prompts can be worth far more than one that removes the same number of scattered errors, and a static audit cannot tell the two apart.

7 Discussion

Conclusion. Whether RLVR inherits or amplifies a base model's reward hacking has no single answer. RLVR is a neutral filter at its KL-regularized optimum, and cross-prompt parameter sharing breaks that baseline in both directions, through a channel proportional to the model's own mass on a wrong answer at the prompts that accept and reject it; removing the sharing removes the channel. In our environment, suppression and amplification are the two ends of one axis with inheritance between them, and at a given step size what moves a setting along it is how much of that mass sits on prompts where the verifier accepts it. For the two literatures, our results suggest a hypothesis rather than establish a reconciliation: the prior studies plausibly sit at different points, since the false positives in the leaky test suites of Zhang [2026] are per-task while the hack in Khalifa et al. [2026] edits the test file and works everywhere, but no estimate of u or u_{eff} exists for those settings. Zhang [2026] finds no per-task growth in rewarded false positives, which is inheritance rather than the suppression at our low-u end, so its released rollouts are a natural place to estimate u_{eff}. The relative-error false positives of Egashira et al. [2026] accept a band around each prompt's own answer and produce a plateau: those bands overlap, so u need not be small, and the model's mass on a near miss sits on prompts that accept it, so u_{eff} may be high. What to audit is not just how many errors a verifier makes or how many prompts share them, but whether the model is wrong the same way the verifier is; for any accepted string, $\bar{V}$ and u_{eff} depend only on the policy and the verifier's mask.

Limitations. Proposition 2 covers only one channel of the drift and assumes the same pass rate on every prompt. Other channels are about as large (Appendix C), so it indicates a direction rather than giving a bound. The setting is small: exact enumeration limits responses to two tokens, so a shared error is one identical string rather than a shared way of going wrong, the leaks are injected through contaminated supervised training rather than discovered by the policy, and the policy is one two-layer transformer with 101,706 parameters. The sign of the drift at low universality depends on the step size, and at $u = 1$ under Adam it varies across seeds. Our reading of prior language model results is a hypothesis that we have not tested on those models.

Future work. The natural next step is to scale up experiments. This would entail measuring u_{eff} and the cross-prompt coherence of accepted wrong responses on a real language model and verifier, both of which can be computed from per-prompt gradients, and to test whether they predict which false positives RLVR amplifies.

AI usage. LLM agents assisted with experiment design and manuscript preparation, but all code and text have been reviewed by the authors.

References

Johannes Ackermann, Michael Noukhovitch, Takashi Ishida, and Masashi Sugiyama. Gradient regularization mitigates reward hacking in reinforcement learning from human feedback and verifiable rewards, 2026. URL https://arxiv.org/abs/2602.18037.

Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting REINFORCE-style optimization for learning from human feedback in LLMs. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12248–12267, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.662. URL https://aclanthology.org/2024.acl-long.662/.

Xin-Qiang Cai, Wei Wang, Feng Liu, Tongliang Liu, Gang Niu, and Masashi Sugiyama. Reinforcement learning with verifiable yet noisy rewards under imperfect verifiers, 2026. URL https://arxiv.org/abs/2510.00915.

Satrajit Chatterjee. Coherent gradients: An approach to understanding generalization in gradient descent-based optimization, 2020. URL https://arxiv.org/abs/2002.10657.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. URL https://arxiv.org/abs/2110.14168.

Marc Dymetman. Binary rewards and reinforcement learning: Fundamental challenges, 2026. URL https://arxiv.org/abs/2605.02375.

Kazuki Egashira, Mark Vero, Jasper Dekoninck, Florian E. Dorner, Robin Staab, and Martin Vechev. Delay, plateau, or collapse: Evaluating the impact of systematic verification error on RLVR, 2026. URL https://arxiv.org/abs/2605.02909.

Tom Everitt, Victoria Krakovna, Laurent Orseau, and Shane Legg. Reinforcement learning with a corrupted reward channel. In *Proceedings of the 26th International Joint Conference on Artificial Intelligence*, IJCAI'17, page 4705–4713. AAAI Press, 2017. ISBN 9780999241103.

Jingchu Gai, Guanning Zeng, Huaqing Zhang, and Aditi Raghunathan. Differential smoothing mitigates sharpening and improves LLM reasoning, 2025. URL https://arxiv.org/abs/2511.19942.

Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In *Proceedings of the 40th International Conference on Machine Learning*, ICML'23. JMLR.org, 2023.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Honghui Ding, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jingchang Chen, Jingyang Yuan, Jinhao Tu, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaichao You, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingxu Zhou, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou,

Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645(8081):633–638, September 2025. ISSN 1476-4687. doi: 10.1038/s41586-025-09422-z. URL http://dx.doi.org/10.1038/s41586-025-09422-z.

Lukas Helff, Quentin Delfosse, David Steinmann, Ruben Härle, Hikaru Shindo, Patrick Schramowski, Wolfgang Stammer, Kristian Kersting, and Felix Friedrich. LLMs gaming verifiers: RLVR can lead to reward hacking, 2026. URL https://arxiv.org/abs/2604.15149.

Yuzhen Huang, Weihao Zeng, Xingshan Zeng, Qi Zhu, and Junxian He. From accuracy to robustness: A study of rule- and model-based verifiers in mathematical reasoning, 2026. URL https://arxiv.org/abs/2505.22203.

Arthur Jacot, Franck Gabriel, and Clément Hongler. Neural tangent kernel: Convergence and generalization in neural networks. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems*, NIPS'18, page 8580–8589, Red Hook, NY, USA, 2018. Curran Associates Inc.

Muhammad Khalifa, Zohaib Khan, Omer Tafveez, Hao Peng, and Lu Wang. Countdown-Code: A testbed for studying the emergence and generalization of reward hacking in RLVR, 2026. URL https://arxiv.org/abs/2603.07084.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015. URL https://arxiv.org/abs/1412.6980.

Tomasz Korbak, Ethan Perez, and Christopher L. Buckley. RL with KL penalties is better viewed as Bayesian inference. In *Conference on Empirical Methods in Natural Language Processing*, 2022. URL https://api.semanticscholar.org/CorpusID:248987624.

Germán Kruszewski, Pierre Erbacher, Jos Rozen, and Marc Dymetman. Whatever remains must be true: Filtering drives reasoning in LLMs, shaping diversity, 2026. URL https://arxiv.org/abs/2512.05962.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step, 2023. URL https://arxiv.org/abs/2305.20050.

Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding R1-Zero-like training: A critical perspective, 2025. URL https://arxiv.org/abs/2503.20783.

Anamika Lochab, Bolian Li, and Ruqi Zhang. Uniform-correct policy optimization: Breaking RLVR's indifference to diversity, 2026. URL https://arxiv.org/abs/2605.00365.

Monte MacDiarmid, Benjamin Wright, Jonathan Uesato, Joe Benton, Jon Kutasov, Sara Price, Naia Bouscal, Sam Bowman, Trenton Bricken, Alex Cloud, Carson Denison, Johannes Gasteiger, Ryan Greenblatt, Jan Leike, Jack Lindsey, Vlad Mikulik, Ethan Perez, Alex Rodrigues, Drake Thomas, Albert Webson, Daniel Ziegler, and Evan Hubinger. Natural emergent misalignment from reward hacking in production RL, 2025. URL https://arxiv.org/abs/2511.18397.

Jincheng Mei, Chenjun Xiao, Csaba Szepesvári, and Dale Schuurmans. On the global convergence rates of softmax policy gradient methods. In *Proceedings of the 37th International Conference on Machine Learning*, ICML'20. JMLR.org, 2020.

Kei Nishimura-Gasparian, Robert McCarthy, and David Lindner. Towards understanding specification gaming in reasoning models, 2026. URL https://arxiv.org/abs/2605.02269.

Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, NIPS '22, Red Hook, NY, USA, 2022. Curran Associates Inc. ISBN 9781713871088.

Jan Peters, Katharina Mülling, and Yasemin Altun. Relative entropy policy search. In *Proceedings of the Twenty-Fourth AAAI Conference on Artificial Intelligence*, AAAI'10, page 1607–1612. AAAI Press, 2010.

Andreas Plesner, Francisco Guzmán, and Anish Athalye. An imperfect verifier is good enough: Learning with noisy rewards, 2026. URL https://arxiv.org/abs/2604.07666.

Ali Rad, Khashayar Filom, Darioush Keivan, Peyman Mohajerin Esfahani, and Ehsan Kamalinejad. Rate or fate? RLV$^{\varepsilon}$R: Reinforcement learning with verifiable noisy rewards, 2026. URL https://arxiv.org/abs/2601.04411.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*, NIPS '23, Red Hook, NY, USA, 2023. Curran Associates Inc.

Yi Ren and Danica J. Sutherland. Learning dynamics of LLM finetuning. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://arxiv.org/abs/2407.10490.

Rulin Shao, Shuyue Stella Li, Rui Xin, Scott Geng, Yiping Wang, Sewoong Oh, Simon Shaolei Du, Nathan Lambert, Sewon Min, Ranjay Krishna, Yulia Tsvetkov, Hannaneh Hajishirzi, Pang Wei Koh, and Luke Zettlemoyer. Spurious rewards: Rethinking training signals in RLVR, 2026. URL https://arxiv.org/abs/2506.10947.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models, 2024. URL https://arxiv.org/abs/2402.03300.

Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward hacking. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, NIPS '22, Red Hook, NY, USA, 2022. Curran Associates Inc. ISBN 9781713871088.

Nisan Stiennon, Long Ouyang, Jeff Wu, Daniel M. Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul Christiano. Learning to summarize from human feedback. In *Proceedings of the 34th International Conference on Neural Information Processing Systems*, NIPS '20, Red Hook, NY, USA, 2020. Curran Associates Inc. ISBN 9781713829546.

Akiyoshi Tomihari. Learning dynamics in RL post-training for language models, 2026. URL https://arxiv.org/abs/2601.04670.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS'17, page 6000–6010, Red Hook, NY, USA, 2017. Curran Associates Inc. ISBN 9781510860964.

Jingkang Wang, Yang Liu, and Bo Li. Reinforcement learning with perturbed rewards, 2020. URL https://arxiv.org/abs/1810.01032.

Zhangchen Xu, Yuetai Li, Fengqing Jiang, Bhaskar Ramasubramanian, Luyao Niu, Bill Yuchen Lin, and Radha Poovendran. TinyV: Reducing false negatives in verification improves RL for LLM reasoning, 2025. URL https://arxiv.org/abs/2505.14625.

Fengkai Yang, Zherui Chen, Xiaohan Wang, Xiaodong Lu, Jiajun Chai, Guojun Yin, Wei Lin, Shuai Ma, Fuzhen Zhuang, Deqing Wang, Yaodong Yang, Jianxin Li, and Yikun Ban. Your group-relative advantage is biased, 2026. URL https://arxiv.org/abs/2601.08521.

Youliang Yuan, Qiuyang Mang, Jingbang Chen, Hong Wan, Xiaoyuan Liu, Junjielong Xu, Jen tse Huang, Wenxuan Wang, Wenxiang Jiao, and Pinjia He. Curing miracle steps in LLM mathematical reasoning with rubric rewards, 2026. URL https://arxiv.org/abs/2510.07774.

Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model?, 2025. URL https://arxiv.org/abs/2504.13837.

Guanning Zeng, Zhaoyi Zhou, Daman Arora, and Andrea Zanette. Shrinking the variance: Shrinkage baselines for reinforcement learning with verifiable rewards, 2026. URL https://arxiv.org/abs/2511.03710.

Chuyifei Zhang. When the reward suite is leaky: A preregistered causal contrast of natural verifier false positives in RLVR, 2026. URL https://arxiv.org/abs/2607.11022.

Rosie Zhao, Alexandru Meterez, Sham Kakade, Cengiz Pehlevan, Samy Jelassi, and Eran Malach. Echo chamber: RL post-training amplifies behaviors learned in pretraining, 2025. URL https://arxiv.org/abs/2504.07912.

Yulai Zhao, Haolin Liu, Dian Yu, Sunyuan Kung, Meijia Chen, Haitao Mi, and Dong Yu. One token to fool LLM-as-a-judge, 2026. URL https://arxiv.org/abs/2507.08794.

A Proofs

A.1 Proposition 1

Fix x and assume $\pi_{\mathrm{ref}}(\cdot \mid x) > 0$, so the KL term is finite and every policy on the path is strictly positive. The objective $\sum_y \pi(y) r(x,y) - \beta \sum_y \pi(y) \log \frac{\pi(y)}{\pi_{\mathrm{ref}}(y)}$ over the simplex is strictly concave for $\beta > 0$; setting the derivative of the Lagrangian to zero gives $\pi^\star_\beta(y \mid x) = \pi_{\mathrm{ref}}(y \mid x) e^{r(x,y)/\beta} / Z_\beta(x)$ with $Z_\beta(x) = \sum_{y'} \pi_{\mathrm{ref}}(y' \mid x) e^{r(x,y')/\beta}$. For y, y' with $r(x,y) = r(x,y')$ the normalizer and the exponential factor cancel in the ratio. Since $r \geq c$ we have $\mathcal{H}(x) \subseteq \mathcal{A}(x)$ and $r \equiv 1$ on $\mathcal{A}(x)$, so

$$\frac{\pi^\star_\beta(\mathcal{H}(x) \mid x)}{\pi^\star_\beta(\mathcal{A}(x) \mid x)} = \frac{e^{1/\beta} \pi_{\mathrm{ref}}(\mathcal{H}(x) \mid x)/Z_\beta(x)}{e^{1/\beta} \pi_{\mathrm{ref}}(\mathcal{A}(x) \mid x)/Z_\beta(x)} = \frac{\pi_{\mathrm{ref}}(\mathcal{H}(x) \mid x)}{\pi_{\mathrm{ref}}(\mathcal{A}(x) \mid x)}$$

for every x, hence ρ and Λ are unchanged. For a multiplicative update $\pi_{t+1}(y \mid x) \propto \pi_t(y \mid x) e^{\eta f(r(x,y))}$ the same cancellation applies at every step.

Excursion of the pooled statistic. Write $a_x = \pi_{\mathrm{ref}}(\mathcal{A}(x) \mid x)$, $\rho_x = \pi_{\mathrm{ref}}(\mathcal{H}(x) \mid x)/a_x$ and $s = e^{1/\beta}$. Then $\pi^\star_\beta(\mathcal{A}(x) \mid x) = s\, a_x / (1 - a_x + s\, a_x)$, and the pooled statistic is $P(s) = \sum_x \rho_x w_x(s) / \sum_x w_x(s)$ with $w_x(s) = s a_x / (1 - a_x + s a_x)$. At $s = 1$ (no RL) $P = \sum_x a_x \rho_x / \sum_x a_x$; as $s \to \infty$ every $w_x \to 1$ provided $a_x > 0$, and $P \to \frac{1}{N} \sum_x \rho_x = \rho(\pi_{\mathrm{ref}})$. The total excursion is therefore

$$P(\infty) - P(1) \;=\; \frac{1}{N} \sum_x \rho_x \;-\; \frac{\sum_x a_x \rho_x}{\sum_x a_x} \;=\; -\frac{\mathrm{Cov}_x(a_x, \rho_x)}{\mathbb{E}_x[a_x]},$$

which is positive whenever pass rate and hack fraction are negatively correlated across prompts. It vanishes when $\mathrm{Cov}_x(a_x, \rho_x) = 0$; constant ρ_x is sufficient but not necessary, since equal a_x at every prompt also gives $P(s) = \frac{1}{N} \sum_x \rho_x$ for every s. The pooled statistic is not an estimator of ρ; it is a deterministic function of the filtering path that happens to converge to ρ in the strong-RL limit.

A.2 Lemma 1

$\nabla_\theta J = \frac{1}{N} \sum_{x'} \sum_{y'} \pi_\theta(y' \mid x') r(x', y') \nabla_\theta \log \pi_\theta(y' \mid x') = \frac{1}{N} \sum_{x'} \mathbb{E}_{y'}[r(x', y') \nabla_\theta \log \pi_\theta(y' \mid x')]$. Since $\mathbb{E}_{y'}[\nabla_\theta \log \pi_\theta(y' \mid x')] = 0$, the expectation equals $\mathrm{Cov}_{y'}(r, \nabla_\theta \log \pi_\theta)$ componentwise. For a fixed vector $v = \nabla_\theta \log \pi_\theta(y \mid x)$, $\langle v, \mathrm{Cov}(r, g) \rangle = \mathbb{E}[r \langle v, g \rangle] - \mathbb{E}[r] \mathbb{E}[\langle v, g \rangle] = \mathrm{Cov}(r, \langle v, g \rangle)$, so the inner product passes inside. □

A.3 Corollary 1

With $\theta = (\theta_x)$ and $\pi_\theta(\cdot \mid x)$ a function of θ_x only, $\nabla_{\theta_{x'}} \log \pi_\theta(y \mid x) = 0$ for $x' \neq x$, so every off-diagonal kernel entry is a sum of products with a zero factor. Moreover $\nabla_{\theta_x} J = \frac{1}{N} \sum_y \pi_\theta(y \mid x)(r(x,y) - V(x)) \nabla_{\theta_x} \log \pi_\theta(y \mid x)$ involves only θ_x, so the flow restricted to block x is autonomous and depends on the verifier only through $r(x, \cdot)$. The argument survives Adam, whose state is per-coordinate and whose momentum is a linear combination of gradients that are already block-supported; it would not survive a global gradient-norm clip, which we do not use. □

Rich-get-richer under vanilla policy gradient. For a single prompt with free logits θ_y and $J = \sum_y \pi_y r_y$, $\partial J / \partial \theta_z = \pi_z (r_z - J)$, so for two responses with $r_1 = r_2 = 1$, $\frac{d}{dt} \log(\pi_1 / \pi_2) = (\pi_1 - \pi_2)(1 - J)$. In the N-prompt setting of Corollary 1 the objective carries a $1/N$ and the baseline is the prompt's own pass rate, giving $\frac{1}{N}(\pi_1 - \pi_2)(1 - V(x))$. The odds between equally rewarded responses are therefore not preserved by vanilla policy gradient even without sharing; they move toward whichever response already has more mass, at a rate that vanishes only as $V(x) \to 1$. This drift is independent of u by Corollary 1.

A.4 Proposition 2

Two parts of the kernel drop out before Definition 3 is applied: a component of $K_\theta((x,h),(x',y'))$ that does not depend on y' contributes zero covariance in (1), and a component that does not depend on h is common to the leak and to the correct answer at prompt x and cancels in the within-prompt log-odds. Substituting the same-string part of the definition, the covariance at x' is

$$\mathrm{Cov}_{y'}[r(x',y'), \kappa_{xx'} \mathbf{1}[y' = h]] = \kappa_{xx'}\, \pi_\theta(h \mid x') \big(r(x',h) - V(x') \big),$$

using $\mathbb{E}_{y'}[r] = V(x')$. Under $V(\cdot) \equiv \bar{V}$ and with $r(x',h) = \mathbf{1}[x' \in U]$,

$$\begin{aligned} \frac{d}{dt} \log \pi_\theta(h \mid x) \Big|_{\mathrm{coh}} &= \frac{1}{N} \Big[\sum_{x' \in U} \kappa_{xx'} \pi_\theta(h \mid x')(1 - \bar{V}) - \sum_{x' \notin U} \kappa_{xx'} \pi_\theta(h \mid x') \bar{V} \Big] \\ &= \frac{W_x}{N} \big(u_{\mathrm{eff}}(x,h) - \bar{V} \big), \end{aligned}$$

with $W_x = \sum_{x'} \kappa_{xx'} \pi_\theta(h \mid x')$ and u_{eff} as in (2) with $\kappa_{xx'}$ in place of $K_{\theta, xx'}$; the sum includes $x' = x$, which is the own-prompt term. The identical computation for the correct answer g_x with $\kappa^g_{xx'} = 0$ off the diagonal leaves only its own term, $\frac{1}{N} \kappa_{s,g} \pi_\theta(g_x \mid x)(1 - \bar{V})$, because $r(x, g_x) = 1$. Subtracting gives (3). Under the further assumptions $x \in U$, $\pi_\theta(h \mid \cdot) \equiv p_h$ and $\kappa_{xx'} = \kappa$ off the diagonal, κ_s on it, $W_x = p_h[\kappa_s + (N-1)\kappa]$ and $u_{\mathrm{eff}} = [\kappa_s + (m-1)\kappa] / [\kappa_s + (N-1)\kappa]$, so

$$\frac{W_x}{N}(u_{\mathrm{eff}} - \bar{V}) = \frac{p_h}{N} \big[\kappa(m-1) - \kappa \bar{V}(N-1) + \kappa_s(1 - \bar{V}) \big] = \kappa p_h (u - \bar{V}) + \frac{(1 - \bar{V})}{N} (\kappa_s - \kappa) p_h,$$

which with the correct answer's own term is the homogeneous form quoted in the proposition. For $x \notin U$ the same computation gives the same leading term with $(1 - \bar{V})$ replaced by $-\bar{V}$ in the correction. □

What the proposition does and does not say. The correction in the homogeneous form is $O(1/N)$ at fixed kernel scales, but it is small relative to the leading term when $(1 - \bar{V}) \big(|\kappa_s - \kappa| p_h + \kappa_{s,g} p_g \big) \ll N \kappa p_h |u - \bar{V}|$, which constrains the kernel, the correct answer's own self-scale, and the distance from the threshold, not N alone; this is sufficient rather than necessary, since the two parts of the correction can cancel, and it says nothing at $u \approx \bar{V}$. The binding factor in our environment is $N\kappa/\kappa_s$, with κ/κ_s measured as the mean cross-prompt gradient cosine of each prompt's own leak: +0.287 at $u = 1$ gives $N\kappa/\kappa_s \approx 18$, but +0.017 at $u = 1/64$ gives about 1.1, where the correction is the same order as the leading term (and at low u the leaks are different strings, so this is not even the same-string scale that the proposition uses). The general form (3) does not need homogeneity of the masses, which is the form we use in Section 6.3, but it still describes only the same-string channel: the residual R in Definition 3, which includes the cross-prompt effect of the correct answers' shared machinery on the leak's logit, is not bounded. Appendix C measures how much of the exact cross-prompt drift the channel accounts for.

Reading the threshold. $\bar{V}$ is the policy's verifier pass rate, observable during training without ground truth, and u_{eff} depends only on the policy and the verifier's mask. A coherent wrong answer is pushed up by the same-string channel when the share of the policy's kernel-weighted mass on it that sits on accepting prompts exceeds the policy's current pass rate. The statement is directional and first-order; it identifies the sign of one channel and the quantity controlling it, not an exact boundary in a model whose kernel is only approximately coherent.

B Environment, Model, and Training

Task. Prompts are all (a, b) with $a, b \in \{2, \ldots, 9\}$ ($N = 64$); the correct response is the two-digit decimal string of ab (all products lie in $[4, 81]$). The response space is all 100 two-digit strings. Reducing the operand set to $\{2, \ldots, 5\}$ gives $N = 16$ for the prompt-set-size sweep. The correct answers are not perfectly prompt-specific (2×6 and 3×4 both give 12), so they carry a universality of their own, $u_g = 0.037$. Under Definition 3 what matters is measured coherence rather than string identity, and we find the correct responses to be near-incoherent across prompts (+0.03) while a universal leak is not.

Leak assignment. Leak strings are drawn without replacement from the 69 two-digit strings that are not the correct answer of any prompt, so a leak is never a correct answer anywhere and every response is unambiguously classified. Prompts are partitioned into N/m blocks; each block receives k leak strings shared by all its prompts. The verifier's static false-positive rate is exactly $k/99$ in every condition, independent of m. Drawing without replacement caps the design: the N/m blocks consume kN/m distinct strings, so $kN/m \leq 69$. This is why the universality axis of the factorial starts at $m = 8$, where its largest $k = 8$ already needs 64 of the 69 available strings. The single-k sweeps are unconstrained.

Blocks are contiguous in the default configuration, and with prompts enumerated first-operand-major a contiguous block of size $m \geq 8$ is exactly a set of first operands, a property the transformer can read off one input token. That would confound universality with how simple the prompt-to-leak map is to represent, so we run the whole sweep a second time with block membership given by a random permutation of prompts and leak strings drawn independently per (m, k, seed). Both versions appear in Figure 2.

What the base model does and does not control. The SFT target puts identical hack mass on every prompt in every condition, but the realized base model is a shared-weight transformer that fits a universal leak map slightly better than an idiosyncratic one, so the measured base hack fraction still varies by 0.015 across universality conditions. We therefore report drifts from each run's own base model, which Proposition 1 makes exactly zero under the filter baseline. In the universality×error-rate factorial the uniform component of the SFT target is spread only over responses that are neither the correct answer nor a leak, which makes total base hack mass exactly ε for every k; otherwise base hack mass would grow with k and the two factors would not be separable. The measured base *fraction* still varies, mean $\rho = 0.129$ with spread 0.080, because diverting the uniform floor also changes the mass on the correct answer. This is the one place where the SFT target differs between experiments, and it is also why the mild decrease of the drift with k in Figure 7 is a property of the design: the same base hack mass is divided among more accepted strings.

Base model. A decoder-only transformer with 2 layers, model dimension 64, 4 heads and a 10-way output head over digits (101,706 parameters) is trained for 600 Adam steps at 3×10^{-3} on the exact target $q(y \mid x) = (1 - \varepsilon)[\lambda_x \mathbf{1}[y = g_x] + (1 - \lambda_x)\text{Unif}(\mathcal{Y})] + \varepsilon\,\text{Unif}(\mathcal{H}(x))$ with $\varepsilon = 0.03$. Competence levels λ_x are an evenly spaced grid on $[0, 0.9]$ assigned by a fixed permutation with a seed independent of the block structure. Because the loss is against an exact target rather than samples, the base distribution is matched across universality conditions without per-condition calibration. Note that the uniform component puts a floor of $(1 - \varepsilon)(1 - \lambda_x)/100$ on every string at every prompt, including a block's leak string on prompts outside the block; that floor is what makes u_{eff} smaller than one for $u < 1$ in the headline sweep (Appendix C).

Tabular policy. Free parameters $w_1 \in \mathbb{R}^{N \times 10}$ and $w_2 \in \mathbb{R}^{N \times 10 \times 10}$ with $\pi(y \mid x) = \text{softmax}(w_1[x])_{y_1} \cdot \text{softmax}(w_2[x, y_1])_{y_2}$. The factorization is complete: its $9 + 10 \cdot 9 = 99$ free directions match the dimension of the simplex over 100 responses. Given the base transformer's

exact log-probabilities we set $w_1 = \log \sum_{y_2} \pi(y \mid x)$ and $w_2 = \log \pi(y \mid x) - w_1$, which reproduces the base distribution to within 10^{-4} in max-norm (asserted at run time).

RL. The default objective is the exact-expectation policy gradient with the exact mean baseline, $\nabla_\theta J = \frac{1}{N} \sum_x \sum_y \pi_\theta(y \mid x)\,[r(x,y) - V(x)]\,\nabla_\theta \log \pi_\theta(y \mid x)$, implemented as a weighted cross-entropy with detached weights. The sampled-GRPO experiment uses group size 8, group-standard-deviation normalization and one inner epoch. Optimizer Adam (default) or SGD, 400 steps, learning-rate grid $\{3 \times 10^{-4}, 10^{-3}, 3 \times 10^{-3}, 10^{-2}, 3 \times 10^{-2}\}$ for Adam and $\{0.3, 1, 3\}$ for SGD, applied identically to both policy classes where a grid is shared (Table 1). The reported operating point is, for each policy class, cell and seed, the smallest learning rate whose final verifier reward is within 10^{-3} of the best. The camera-ready applies this one rule to every experiment, including verifier hardening, which the earlier version selected by strict argmax.

The mask-only controls (E9, E9b). One checkpoint per seed serves every universality condition. Its supervised target is contaminated with a shared string h and a per-prompt string s_x at every prompt, in identical proportion in all conditions, so the initial policy, the probability on each leak string and the initial tangent kernel are bit-identical across the sweep; the base probability on h agrees to six decimals. The verifier then accepts h on the m prompts in U and, in E9, s_x on the rest, giving exactly N accepted wrong pairs and a static false-positive rate of exactly $1/(|\mathcal{Y}| - 1)$ in every condition. E9b accepts nothing wrong outside U, so every accepted wrong string is h and its per-prompt base mass is identical across conditions, at the cost of a static false-positive rate of $u/99$ and a base hack fraction that scales with u (prompts that accept nothing wrong have $\rho_x = 0$). U is nested across m. The residual gradient in E9 is that the accepted string's base mass rises from 0.0098 to 0.0149 with u, because a shared-parameter SFT represents one string demonstrated at every prompt better than N distinct ones; the per-prompt arm responds to that gradient through prompt-local dynamics, which Corollary 1 permits since it is inherited rather than generated. Reweighting the two families in the supervised target until their realized probabilities match closes the mean gap but not the per-prompt gap, because the shared network cannot represent an arbitrary per-prompt split; we therefore bracket the confound with two designs rather than eliminate it.

Per-prompt weight copies (E10). N independent copies of the same transformer, each initialized from the shared base checkpoint and trained on its own prompt alone with the same objective and optimizer, on the three smallest learning rates of the shared arm's grid. Copy x only ever evaluates prompt x, so training the copies in one loop is equivalent to training them separately. The matched shared-parameter arm is E1 restricted to the same four universality conditions and the same ten seeds, with slopes averaged per seed, which is why its slope (+0.0210) differs slightly from the seven-condition figure (+0.0177).

Verifier hardening (E6). The base model is contaminated with both leak families, one universal string and one idiosyncratic string per prompt. Three verifiers are then deployed from the same checkpoint: unpatched, universal leak removed, and all idiosyncratic leaks removed. The two patches remove exactly N accepted (prompt, wrong-response) pairs each. Both arms are trained at two learning rates (10^{-3} and 3×10^{-3} for the shared arm, thirty times that for the tabular arm, as in the KL sweep), and the operating point is chosen by the paper's standard rule, which picks 0.001 for the shared arm. Every statistic in Section 6.5 is a paired within-seed difference between the two patched verifiers over 10 seeds.

The threshold probe (E11). For each probed prompt x that accepts a leak h we compute by autograd the exact drift of $\log \pi_\theta(h \mid x) - \log \pi_\theta(g_x \mid x)$ under gradient flow, split into its own-prompt and cross-prompt parts, and the gradients of $\log \pi_\theta(h \mid x')$ and $\log \pi_\theta(g_x \mid x')$ at every prompt x', which give the same-string kernel rows $K_{\theta,xx'}$ and the probe's estimate of u_{eff} (Equation (2), with the own term included). The coherent-channel prediction is the $y' = h$ and $y' = g_x$ terms of Lemma 1 summed over $x' \neq x$, with the measured masses and kernel entries; the homogeneous formula uses the mean off-diagonal entry as κ, the diagonal entry as κ_s, and the leak's mean probability on accepting prompts as p_h. The probe runs at the base and at SGD steps 0, 3, 6, 10, 15, 20, 40 with step size 0.3, on 8 accepting prompts per condition at $m \in \{1, 4, 16, 64\}$ and 3 seeds, and once at the frozen blind base of E9b for the same m.

Full specification. Prompts are serialized as the token sequence $[\texttt{BOS}, a, b, y_1]$ over a vocabulary of 11 symbols, with learned token and positional embeddings, a causal mask, pre-norm blocks, 4 heads, feed-forward width 256 with GELU, final `LayerNorm`, and a 10-way output head shared across the two response positions. Embeddings are initialized $\mathcal{N}(0, 0.02^2)$ and the remaining parameters use PyTorch defaults. Optimization is full-batch over all N prompts with no minibatching and no gradient clipping. Adam uses $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$ and zero weight decay; SGD uses no momentum. A run's seed sets the transformer initialization, the leak-string draw and, in E4, the rollout sampling; the competence permutation uses a separate fixed seed. E4 draws $G = 8$ responses per prompt at temperature 1 from the enumerated distribution, normalizes advantages by the group standard deviation (plus 10^{-4}), and takes one inner epoch with no importance ratio, clipping or reference-KL term. E5 adds $-\beta\,\mathrm{KL}(\pi_\theta(\cdot \mid x)\|\pi_{\mathrm{ref}}(\cdot \mid x))$ computed exactly against the frozen base.

Measurement. ρ, Λ and true accuracy are computed from the exact enumerated distribution over all 100 responses at all N prompts at every logged step; the only stochasticity is over seeds. The gradient decomposition and coherence probes average over fixed random subsamples of prompts, for cost. Per-prompt log-odds are clipped to ± 20 before averaging, so a prompt whose leak mass has collapsed cannot dominate Λ; in the headline Adam sweep this binds on a median of 1.6% of prompts, and the ordering of every reported series is unchanged at clipping thresholds of 20, 40, 60 and 200. The final metric of every run is recomputed in float64 from stored per-prompt masses. The two null-model checks of Section 5 are floating-point zeros: exact mirror descent preserves ρ to 8.6×10^{-9} and Λ to 9.6×10^{-8} over 400 steps, and the KL-regularized optimum preserves ρ to 8.6×10^{-9} across $\beta \in$[0.05, 2.0]. The two agree to the last bit by construction, since 400 steps of mirror descent at $\eta = 0.05$ reach exactly $\pi_0 e^{20r}$, the $\beta = 0.05$ endpoint of the KL grid. The per-prompt policy's cross-prompt kernel is exactly 0, against 0.098 of the self-kernel for the transformer.

	sweep	seeds	learning rates	notes
E1	universality, contiguous blocks	10	5, shared by both arms	headline sweep
E2	universality, permuted blocks	10	3, shared by both arms	base spread 0.0103
E3	universality $\times$ error rate	10	1 pair (10^{-3} / 3×10^{-2})	clean SFT floor
E4	sampled GRPO	10	3, shared by both arms	$G = 8$, std norm.
E5	KL sweep	10	1 pair (10^{-3} / 3×10^{-2})	6 values of β
E6	verifier hardening	10	2 pairs	standard rule
E7	prompt-set size $N \in \{16, 64\}$	10	3, shared by both arms	tests u versus m
E8	plain SGD	10	3, shared by both arms	robustness
E9	mask-only, constant error rate	10	5, shared by both arms	one frozen base per seed
E9b	mask-only, constant leak mass	5	5, shared by both arms	one frozen base per seed
E10	per-prompt weight copies	10	3	same architecture
E11	threshold probe	3	SGD 0.3	exact vs. coherent drift

Table 1: What was run in each experiment. Where a grid is shared by both arms the operating point is the smallest learning rate within 10^{-3} of the best verifier reward, chosen independently for each arm. E2, E3, E5, E6, E7 and E11 were run for the camera-ready from base models regenerated with the released code (Appendix E).

C The Threshold Probe

Two questions remain open after Section 4: whether Proposition 2 predicts anything about the sweeps, given that with raw u it predicts suppression below $u \approx \bar{V} \geq 0.44$, and whether the instantaneous prediction can be compared with the exact drift before the verifier pass rate saturates. Figure 4 (bottom right), Figure 6 and the numbers below answer both.

Where the mass sits. Equation (2) has two readings (Figure 4, bottom right). The mass-only reading sets every kernel entry to one and asks what share of the policy's mass on h sits on accepting prompts; the kernel-weighted reading includes the entries, and at small m is dominated by the own-prompt entry κ_s, which is several times the cross-prompt entries. In the headline bases the mass-only reading is 0.09, 0.23, 0.68 and 1.00 at $u = 1/64, 1/16, 1/4$ and 1, and the kernel-weighted reading is 0.65, 0.64, 0.94 and 1.00. The supervised stage puts about 0.0328 of mass on an idiosyncratic leak at the one prompt that accepts it and 0.0054 elsewhere, so the accepting prompts hold several times their proportional share of the sum. In the blind base the mass on h is nearly uniform over prompts,

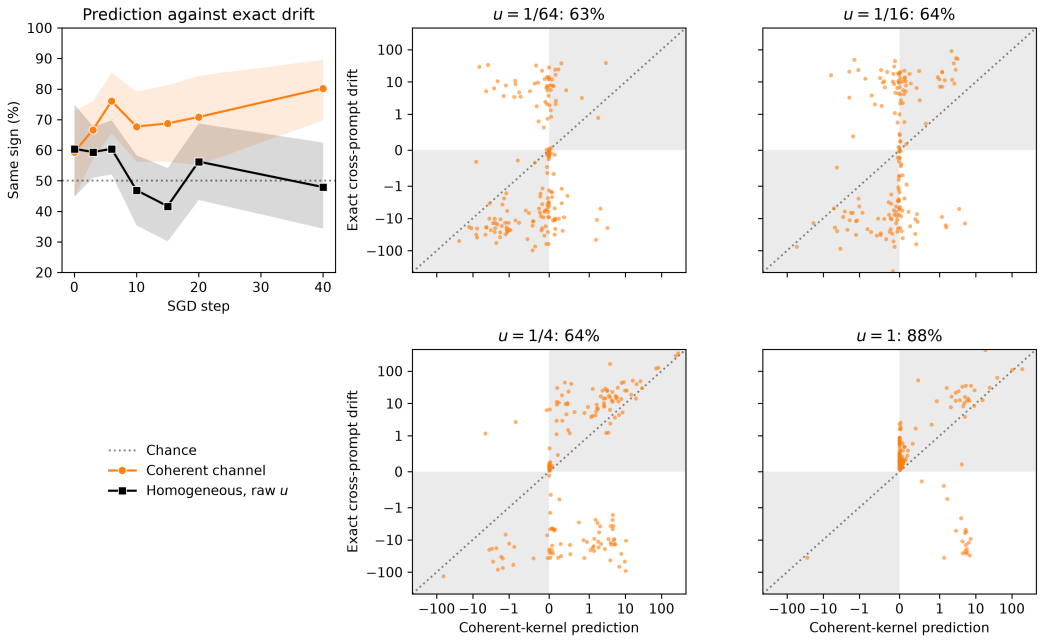


Figure 6: The threshold probe. Left: along plain SGD, the fraction of probed prompts at which the coherent-channel prediction, and the homogeneous raw-u formula, have the same sign as the exact cross-prompt log-odds drift, with bootstrap 95% bands over the (seed, universality) runs. Right: the coherent-channel prediction against the exact drift for every probed prompt at every step, one panel per universality, on symmetric log axes. Shaded quadrants are where the two agree in sign, and each title gives the share of points there.

so the mass-only reading is close to u (0.01, 0.06, 0.25, 1.00) and the kernel-weighted reading is 0.08, 0.19, 0.33 and 1.00. At $u = 1$ the two bases represent h with similar cross-prompt coherence ($+0.294$ in the headline base against $+0.255$ in the blind base), and the blind base carries less mass on it (0.0355 against 0.0151) because its contamination budget is split between two families. This is the representation difference that has to be identified before the blind-base controls can be interpreted: at $u = 1$ the coherence is similar and the mass is lower, which works against amplification there; at $u < 1$ the difference is where the mass sits, not how coherently h is represented. The weights $w_{x'}$ in Equation (2) are positive for most entries: 13% of the same-string kernel entries at the headline bases are negative, carrying 9% of the absolute weight, so u_{eff} is a fraction in $[0, 1]$ to a good approximation there.

Predicted against exact drift. Along SGD the verifier pass rate of the universal-leak condition reaches 0.994 by step 40, while the idiosyncratic condition is still at 0.848, so the window in which $u_{\text{eff}} - \bar{V}$ changes sign is short at high u and long at low u. Over all probed (condition, step) means the coherent-channel prediction agrees in sign with the exact cross-prompt log-odds drift at 93% of points (28 points), and the homogeneous leading-order formula with its $1/N$ correction at 71%; per probed prompt the agreement is 70% and 53% over 672 prompts. The median ratio of the residual to the exact drift is 0.9. The channel the proposition isolates is therefore real, in the sense that its sign and the sign of the exact cross-prompt drift agree well above chance at the level of condition means and above chance per prompt, while the raw-u formula is at chance per prompt. But it is not the whole cross-prompt drift: the residual R of Definition 3, dominated by the shared machinery that raises the correct answers on every prompt, is of the same order as the channel itself. We present the proposition accordingly, as a statement about one channel and the quantity that sets its sign, and we use its heterogeneous form only to interpret the difference between the headline sweep and the blind-base controls.

D Additional Results

Every learning rate. Table 2 lists, for each training setup, learning rate and arm, the slope of $\Delta\rho$ per doubling of m with its bootstrap interval, the drift at the two ends of the sweep, the number of seeds in which the idiosyncratic leak is suppressed and the universal leak amplified, and the final verifier reward at $u = 1$.

setup	arm	LR	slope	95% CI	$\Delta\rho(1/64)$	$\Delta\rho(1)$	neg. at $1/64$	pos. at 1	reward at 1
Adam	shared	0.0003	$+0.0075$	$[+0.0030, +0.0121]$	-0.103	-0.051	10/10	2/10	0.997
Adam	shared	0.001	$+0.0176$	$[+0.0086, +0.0296]$	-0.102	$+0.028$	10/10	5/10	1.000
Adam	shared	0.003	$+0.0996$	$[+0.0753, +0.1251]$	-0.098	$+0.600$	10/10	10/10	1.000
Adam	shared	0.01	$+0.1287$	$[+0.1151, +0.1399]$	$+0.210$	$+0.869$	0/10	10/10	1.000
Adam	shared	0.03	$+0.0456$	$[+0.0246, +0.0632]$	$+0.331$	$+0.651$	0/10	10/10	0.319 (diverged)
Adam	per-prompt	0.0003	$+0.0000$	$[-0.0000, +0.0001]$	-0.000	-0.000	8/10	2/10	0.527
Adam	per-prompt	0.001	$+0.0001$	$[-0.0000, +0.0002]$	$+0.001$	$+0.002$	2/10	10/10	0.644
Adam	per-prompt	0.003	$+0.0004$	$[+0.0000, +0.0009]$	$+0.020$	$+0.021$	0/10	10/10	0.870
Adam	per-prompt	0.01	$+0.0020$	$[+0.0007, +0.0032]$	$+0.110$	$+0.116$	0/10	10/10	0.982
Adam	per-prompt	0.03	$+0.0028$	$[+0.0006, +0.0052]$	$+0.161$	$+0.164$	0/10	10/10	0.997
Sampled GRPO	shared	0.0003	$+0.0094$	$[+0.0047, +0.0145]$	-0.102	-0.032	10/10	3/10	0.996
Sampled GRPO	shared	0.001	$+0.0942$	$[+0.0743, +0.1135]$	-0.098	$+0.623$	10/10	10/10	0.998
Sampled GRPO	shared	0.003	$+0.1675$	$[+0.1594, +0.1742]$	-0.077	$+0.869$	10/10	10/10	1.000
Sampled GRPO	per-prompt	0.0003	-0.0000	$[-0.0001, +0.0000]$	-0.005	-0.005	10/10	0/10	0.509
Sampled GRPO	per-prompt	0.001	-0.0001	$[-0.0002, +0.0000]$	-0.016	-0.016	10/10	0/10	0.584
Sampled GRPO	per-prompt	0.003	-0.0002	$[-0.0005, +0.0001]$	-0.042	-0.041	10/10	0/10	0.770
Plain SGD	shared	0.3	$+0.1834$	$[+0.1773, +0.1889]$	-0.077	$+0.867$	10/10	10/10	0.987
Plain SGD	shared	1	$+0.0982$	$[+0.0718, +0.1161]$	$+0.153$	$+0.771$	0/10	9/10	0.906
Plain SGD	shared	3	-0.0008	$[-0.0197, +0.0158]$	$+0.274$	$+0.299$	0/10	9/10	0.119 (diverged)
Plain SGD	per-prompt	0.3	-0.0001	$[-0.0003, +0.0000]$	-0.017	-0.017	10/10	0/10	0.534
Plain SGD	per-prompt	1	-0.0004	$[-0.0008, +0.0000]$	-0.045	-0.045	10/10	0/10	0.651
Plain SGD	per-prompt	3	-0.0006	$[-0.0014, -0.0000]$	-0.076	-0.077	10/10	0/10	0.837

Table 2: The universality effect at every learning rate. Slopes are per doubling of m; intervals are bootstrap 95% over ten seeds. The last column is the final verifier reward at $u = 1$ for that arm, with divergent settings marked; per-prompt rewards below 0.9 are still rising at step 400.

Universality against error rate (E3). Figure 7 crosses universality with the static false-positive rate, both swept eightfold, with the total base hack mass held at ε for every k so that only the verifier changes; Figure 8 shows the same data seed by seed on a common scale. A variance decomposition over the 10-seed cells attributes 43% of the variance in $\Delta\Lambda$ to universality and 12% to the false-positive rate. Averaged over k, $\Delta\Lambda$ runs from -7.19 to -4.15; averaged over u, only from -5.54 to -6.85. The mild decrease with k is a property of this design, in which accepting more strings spreads a fixed base mass thinner (Appendix B), and we do not read it as a general prediction about audits. The robust content is the comparison of magnitudes: the false-positive rate is the weaker variable.

Prompt-set size (E7). At fixed u the leading term of the homogeneous form of (3) carries no dependence on the number of prompts; the $1/N$ remainder differs fourfold between $N = 16$ and $N = 64$. Repeating the sweep at both sizes (10 seeds) gives, at $u = 1$, -0.024 ± 0.053 against $+0.021 \pm 0.028$, whereas at the same block size $m = 16$ (so $u = 1$ for the small set and $u = 1/4$ for the large one) they sit at -0.024 against -0.081. Across the 5 matched values of u the two sizes agree to within 0.016 at the three smallest values and differ by at most 0.045, at $u = 1$, where the seed spread is widest. Within the precision of ten seeds the controlling variable is the fraction u, not the count m.

Figure 9 collects the remaining robustness views: the universality effect across the whole Adam grid, hack-fraction trajectories at the two extreme conditions for both policy classes, the sweep at two prompt-set sizes, and true accuracy under the KL penalty of Section 6.4.

E Reproducibility

Every experiment runs on CPU; no GPU is used. The code, the raw result files and the figure code that regenerates every figure and every number quoted in the text are released together at `https://github.com/Rohan-Pandey1729/RLVR-Reward-Hacking`. The macro file that holds the numbers is written by the figure code, so nothing is transcribed by hand except settings (the step

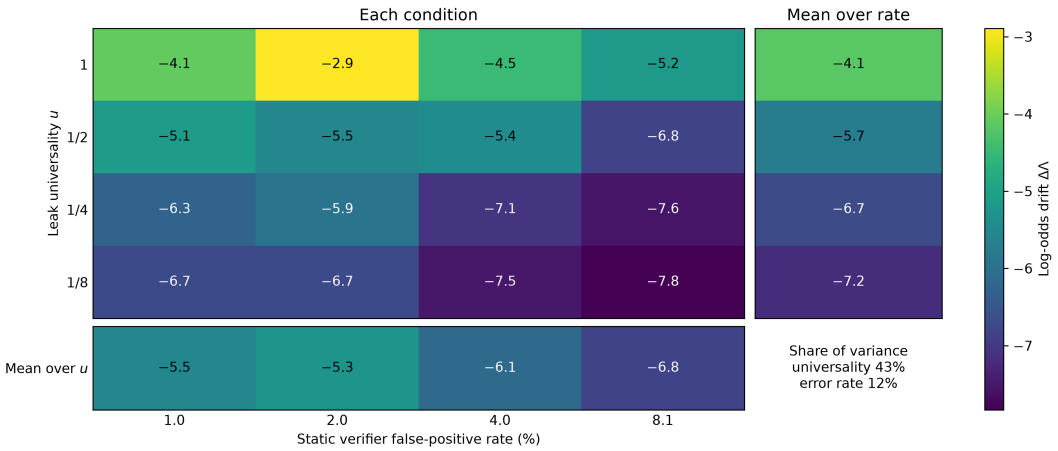


Figure 7: The factorial as a field. Each cell is one (universality, error rate) condition; darker is a leak more strongly suppressed relative to the filter baseline. The right column and bottom row are the marginal means on the same color scale. Total base hack mass is fixed at ε in every cell, so the horizontal trend reflects the same mass spread over more accepted strings rather than a general property of audits.

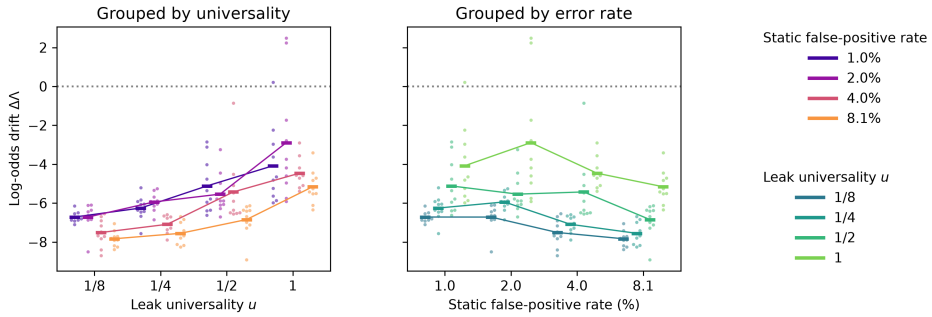


Figure 8: The factorial seed by seed, common y-scale: dots are seeds, bars cell means, and lines join the means of one color. Left: log-odds drift grouped by universality, one color per false-positive rate. Right: the same data grouped by the static false-positive rate, one color per universality. Varying the accepted wrong responses per prompt from 1–8 moves the false-positive rate over 1.0–8.1% and the drift from -5.54 to -6.85; the same eightfold range of universality moves it from -7.19 to -4.15.

count, the contamination rate, the GRPO group size) and a few diagnostics quoted inline (the clipping rates and the $N\kappa/\kappa_s$ values).

Three levels of reproduction apply. From an archived base checkpoint, on the same machine and code revision, the RL stage is bit-reproducible in weights and in every reported metric. Regenerating a base by rerunning the supervised stage is deterministic, but a base rebuilt under a different version of the numerical libraries differs from an archived one by up to about 2×10^{-2} on the base metrics. A run from scratch therefore reproduces the reported drifts and the conclusions drawn from them, which are an order of magnitude larger, but not every archived value digit for digit.

Provenance for the camera-ready: E1, E4, E8, E9, E9b and E10 are the archived runs of the accepted version. E2, E3, E5, E6 and E7, which the accepted version reported at three seeds, were re-run from scratch at ten seeds (E6 with the paper's standard operating rule and both learning rates stored; E2 without the learning rate 3×10^{-2}, which diverged in every seed of the earlier version and was never selected), and E11 is new; these use base models regenerated with the released code, and the E6 base models of the earlier version were not retained. Seeds 0–4 of E1 come from an earlier revision whose configuration records carry fewer logging fields, with no training setting differing; their stored base models are bit-identical and the current code reproduces that half to 10^{-8}. The $m = 1$ condition of E1 ran past the reported seed budget during an attempt to raise the budget for every condition; the

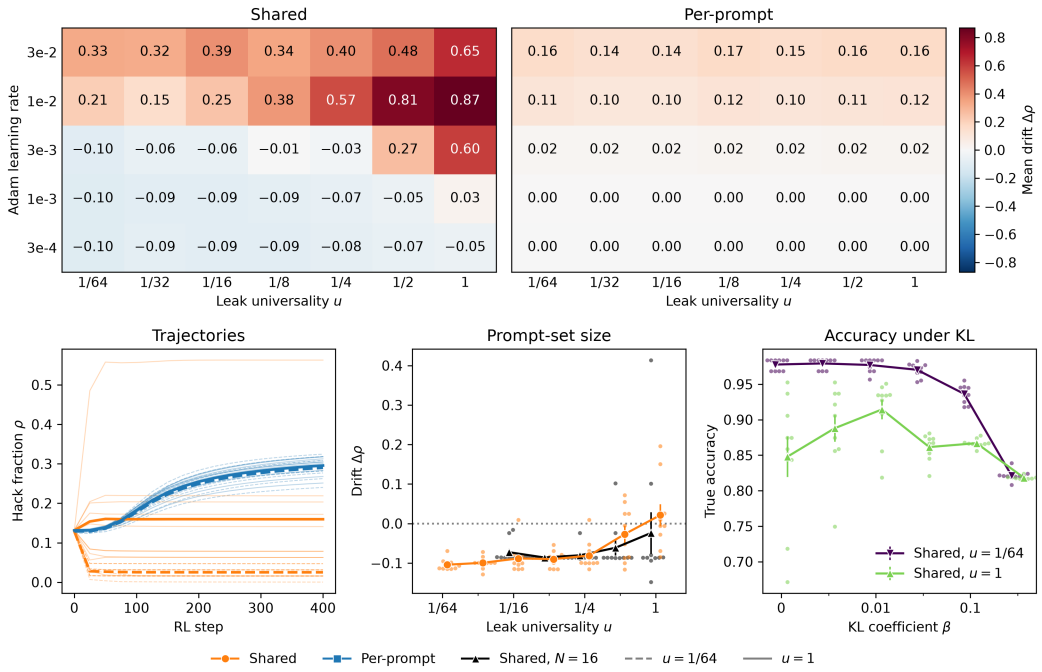


Figure 9: Top: the universality effect across the whole Adam learning-rate grid for both policy classes, as the mean drift over ten seeds on one color scale. Bottom left: hack-fraction trajectories for the extreme universality conditions, both policy classes, with thin lines for seeds; the per-prompt curves coincide. Bottom center: the sweep repeated at two prompt-set sizes, plotted against u, with $N = 64$ in orange. Bottom right: true accuracy at the two ends of the sweep as the KL coefficient grows. In the bottom center and right panels small dots are seeds and markers their means ± 1 s.e.m.

figure code caps every load at ten seeds, uniformly and independently of results, so the design is balanced and the excess runs enter no reported statistic.

The reviewed version of this paper summarized the mask-only controls (E9, E9b) by a linear slope in $\log_2 m$ and concluded that sharing at RL time is not sufficient. We withdraw that conclusion: read per universality value rather than through a fitted slope, the shared policy's curve is a step, not a line (Figure 4, bottom).

F Additional Related Work

RLVR as filtering. The KL-regularized optimum as an exponential tilt of the reference model is well established [Korbak et al., 2022, Gai et al., 2025, Kruszewski et al., 2026, Dymetman, 2026, Lochab et al., 2026, Rafailov et al., 2023, Yue et al., 2025]. That literature reads the invariance as a statement about diversity among correct answers; we read it as a constraint on the composition of the accepted set and use it as our null model, with the rich-get-richer behavior of softmax policy gradient among tied responses [Mei et al., 2020, Lochab et al., 2026] as a control.

Verifier error. Wang et al. [2020] and Cai et al. [2026] show that correcting for reward noise helps, not that uncorrected training is robust. Plesner et al. [2026] measure noise tolerance directly, Rad et al. [2026] map a phase transition in the Youden index, and spurious rewards occasionally help particular models [Shao et al., 2026]. Audits of verifier false positives [Zhao et al., 2026, Huang et al., 2026, Xu et al., 2025, Yuan et al., 2026] report gross error rates and leave the alignment of the errors across prompts unmeasured.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and the list of contributions in the introduction state the claims, their scope (a controlled environment with a small transformer) and their main caveats, such as the dependence on step size. Each contribution cites the section that supports it.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: The "Limitations" paragraph of the discussion covers the small scale, the single architecture, the injected leaks and the untested reading of prior results. Appendices A and C state what Proposition 2 does not cover.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: Propositions 1 and 2, Lemma 1 and Corollary 1 state their assumptions, and Appendix A proves each one. Proposition 2 covers only the coherent part of the kernel, and the paper states that the residual is not bounded.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Section 5, Appendix B and Table 1 give the task, verifier, model, training procedure, hyperparameters and seeds of every experiment. Appendix E states how closely a rerun reproduces the reported values.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

(d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: The code, raw result files and figure code are public on GitHub. The README gives the environment and the exact command for every experiment, and the task data are generated by the code.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: Section 5 gives the setting and the rule for selecting the learning rate, and Appendix B and Table 1 give the optimizers, learning-rate grids, step counts and remaining hyperparameters. There is no train/test split, because every metric is computed exactly over all prompts.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

Justification: Results are reported over seeds, which set the initialization, the leak-string draw and, for sampled GRPO, the rollout sampling, with ten seeds in all but two experiments (Table 1). Figures show individual seeds with means ± 1 s.e.m. or bootstrap 95% intervals, as each caption states, and the key comparisons use paired within-seed differences.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: All experiments run on CPU with no GPU (Appendix E), and a process needs under 1 GB of memory. On one core of a laptop CPU (Intel Core i7-1360P), a base model takes one to two minutes to build and a 400-step run of both policies one to three minutes, which puts the reported experiments on the order of 100 core-hours, and the project also included preliminary runs, among them the three-seed versions that the ten-seed reruns replaced.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The work trains a small transformer on a synthetic multiplication task and involves no human subjects or personal data. It conforms to the NeurIPS Code of Ethics.

   Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [No]

    Justification: The paper has no broader-impacts discussion. It is foundational work on a small transformer and a synthetic task, with no deployed system and no direct path to a negative application, and its practical message is aimed at reducing reward hacking.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
    - The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
    - If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The release contains only code and result files for a small transformer trained on a synthetic multiplication task, with no model weights, pre-trained language model, image generator or scraped data, so it poses no such risk.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.

- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: No existing dataset or model is used, because the task is generated by our own setup. We use popular Python libraries with permissive licenses.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: The new assets are the code and raw result files in the repository above. Its README documents what is measured, the file layout, how to rerun each experiment and how to rebuild the paper, and no data from people are involved.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: The paper does not involve crowdsourcing or research with human subjects.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.

- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: The paper does not involve crowdsourcing or research with human subjects.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [Yes]

    Justification: The end of the discussion states that LLMs assisted with experiment design and manuscript preparation. No LLM is part of the method or the experiments, whose only model is a small transformer trained from scratch.

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.